

How to Turn the Current Project into a Full Marine Autonomy Simulator Platform

A practical Unity-first implementation guide based on the existing MMG + NMPC + PPO hybrid yacht docking project

Prepared for Unity 6000.3.9f1

This guide compiles and expands the simulator-platform recommendations into a structured implementation document aimed at a first-time Unity user. It keeps the current architecture—Python as the source of truth for vessel dynamics and control, Unity as the scenario and visualization engine—and shows how to evolve it into a modular, reusable marine autonomy simulator similar in spirit to VRX, but built around Unity and the present codebase.

Important coordinate convention. Keep one convention throughout the simulator. In Unity, vessel forward motion is +X, lateral motion is +Z, vertical is +Y, and yaw is rotation about +Y. On the Python side, keep the planar marine state explicit and perform all frame conversion in one bridge layer only.

Why this is already a strong foundation

The project already contains the correct architectural seed for a simulator platform: a physics-based MMG vessel model in Python, CasADi-based NMPC, PPO-based DRL, a Unity 3D marina environment, and a TCP/IP communication bridge between Python and Unity. The existing implementation is also already separated into plant/control, learning, and visualization layers, which is exactly the kind of structure that can be expanded into a real simulator platform.

Project-specific architecture anchors used in this guide:

- Python computes vessel dynamics and control, while Unity acts as a rendering and scenario engine rather than as the hydrodynamics engine.
- State streaming is deterministic and thread-safe through TCP reception, background listening, and main-thread dispatching in Unity.
- The marina scene, berth zones, dock triggers, and obstacles were already exported from Unity into structured JSON for DRL scenario reconstruction in Python.
- The current stack already supports layered static and dynamic obstacle groups, which is the correct starting point for a simulator-grade scenario system.

This guide is aligned with the uploaded thesis and presentation, especially the sections describing the Unity–Python co-simulation architecture, scene export pipeline, layered static/dynamic obstacle setup, and the MMG–NMPC–PPO hybrid control stack.

Document map

- Part I — Core simulator principles
- 1. Define the simulator as a layered architecture
- 2. Convert the current scene into a scenario system

- 3. Support multiple vessels, not only one yacht
- 4. Formalize a simulator core API
- 5. Add a simulator manager structure in Unity
- 6. Separate simulation time from rendering time
- 7. Add sensor realism
- 8. Expand the environment model beyond calm water
- 9. Build mission modes instead of one docking task
- 10. Add evaluation and benchmarking
- 11. Create a training interface for RL
- 12. Upgrade Unity into a simulator UI
- 13. Keep Crest, but use it intentionally
- 14. Reorganize folders and modules
- 15. Define the practical development roadmap
- Part II — Unity Editor implementation steps
- Part III — Final checklists and recommended next actions

Part I — Core simulator principles

A simulator platform is different from a one-off experiment. A project proves that one controller can solve one task in one environment. A simulator platform provides reusable architecture, repeatable scenario definition, multiple controller options, data logging, replay, and enough structure that future experiments can be added without rewriting the whole project.

1. Define the simulator as a layered architecture

Why this matters

Your current project already has the correct separation at a conceptual level. The next step is to make that separation explicit in code, scene organization, and data flow. Once the platform is layered, you can replace a controller, add a sensor model, or add a second vessel without breaking unrelated parts of the simulator.

Recommended layers

Layer	What it owns	What it should not own
World and scenario layer (Unity)	Marina geometry, docks, berths, landmarks, cameras, water, traffic prefabs, sensor visuals, UI	Hydrodynamic truth or controller optimization
Physics layer (Python first)	MMG 3-DOF dynamics, actuator models, disturbances, numerical integration, model parameters	Scene rendering or visual-only logic
Controller layer	NMPC, PPO policy, supervisor, heading hold, waypoint following, docking logic	Unity scene object management
Perception layer	GPS, IMU, AIS, radar abstractions, noise and delay models	Direct rendering concerns

Evaluation layer	Metrics, logs, success/failure, benchmark summaries, replay data	Real-time scene manipulation
Visualization and replay layer (Unity)	Live playback, historical replay, trajectory overlays, operator screens	Model fitting and optimization

Theoretical note

In marine autonomy, the simulator should preserve a clean distinction between the plant and the viewer. That matters because controllability studies, docking validation, and hybrid-controller comparisons become ambiguous if Unity physics or hidden GameObject forces start contributing to the motion. Your thesis already chose the scientifically safer architecture: Unity receives pose updates and does not secretly compute vessel maneuvering dynamics.

Actionable Unity Editor steps

- Open the project and create top-level folders under Assets: Scenes, Prefabs, Scripts, Materials, ScriptableObjects, UI, Resources, Art, and ThirdParty.
- Create an empty GameObject in the main scene named `_App`. Reset its Transform. This becomes the parent for all global simulator managers.
- Create child GameObjects under `_App` named `SimulationManager`, `ScenarioManager`, `TrafficManager`, `SensorManager`, `PythonBridgeManager`, `ReplayManager`, and `UIManager`.
- Create another empty root object named `WorldRoot`. Under it, create `EnvironmentRoot`, `MarinaRoot`, `TrafficRootStatic`, `TrafficRootDynamic`, `SensorVisualsRoot`, and `EffectsRoot`.
- Move all existing docks, piers, buoys, and static ships into these roots. This is your first architectural cleanup task.

Implementation rule. If you are unsure where a new feature belongs, ask a simple question: does it change the vessel physics, the scenario definition, or just the view? That answer usually tells you whether the code belongs in Python, a Unity manager, or a visual component script.

2. Convert the current scene into a scenario system

Why this matters

At the moment, the marina is a designed scene. That is good for testing, but a simulator platform needs reusable scenario definitions. A scenario is a configuration of the world: vessel start state, goal, obstacle set, dynamic traffic behavior, environment settings, and active controller. If these remain embedded only in one scene, experiments become hard to reproduce and compare.

Scenario design principle

Separate reusable geometry from variable scenario data. The dock meshes and lighthouse can remain prefabs. The positions of buoys, berth goals, dynamic traffic routes, weather presets, and controller choice should become data that can be loaded, saved, and reused.

Recommended Unity approach

- Turn repeated marina elements into prefabs: DockSegment, PierSegment, BerthZone, Buoy, StaticVessel, DynamicVessel, WaypointMarker, Lighthouse.
- Keep one or more base scenes for geometry only, for example Marina_Base and Marina_Extended.
- Store scenario settings in ScriptableObjects or JSON. For a beginner, ScriptableObjects are easier inside the Unity editor; JSON is better when Python also needs to read the same scenario file.
- Add a ScenarioDefinition asset that contains own-ship spawn pose, goal pose, active obstacle objects, traffic routes, wind/current presets, and controller mode.

Unity Editor steps

- In the Hierarchy, select a dock section object and drag it into Assets/Prefabs/Environment to create a prefab. Repeat this for piers, buoys, and obstacle vessel objects.
- Create a new folder Assets/ScriptableObjects/Scenarios.
- Create a C# script named ScenarioDefinition.cs that derives from ScriptableObject. Add fields for startPose, goalPose, staticObstacleReferences, dynamicRouteReferences, environmentPreset, and controllerMode.
- In the Project window, right-click ScriptableObjects/Scenarios and create several scenario assets, such as Scenario_Marina_Dock_01 and Scenario_Marina_Avoid_01.
- Add a ScenarioManager script to the ScenarioManager GameObject under _App. Expose a public field for the active ScenarioDefinition asset.
- In Play mode, ScenarioManager should spawn or enable the correct prefabs under TrafficRootStatic and TrafficRootDynamic according to the selected scenario.

Python-side step

Ensure the same scenario can be exported or mirrored into Python. Because you already export obstacles and dock goals from Unity into JSON, the scenario system should build on that pattern rather than replace it.

3. Support multiple vessels, not only one yacht

Why this matters

A marine autonomy simulator becomes substantially more valuable when it supports other traffic. Not every moving vessel needs the same fidelity as the controlled yacht. The platform should support different fidelity tiers so that experiments scale without becoming too expensive or too difficult to maintain.

Recommended fidelity tiers

Tier	Use case	Recommended model
Tier 1	Controlled own-ship	Full MMG 3-DOF dynamics in Python, full logging, full controller integration
Tier 2	Important traffic vessels that affect decisions	Simplified 3-DOF or reduced-order kinematic model with route following
Tier 3	Background vessels for scene realism	Simple waypoint or local-axis mover scripts in Unity

Unity Editor steps

- Create three prefabs for traffic categories: TrafficVessel_Full, TrafficVessel_Medium, and TrafficVessel_Background.
- For full and medium vessels, create a common component script such as VesselIdentity.cs with fields for vesselId, vesselType, and controlMode.
- Group moving non-player vessels under TrafficRootDynamic.
- For background actors, add a simple route follower script that moves the object along waypoint transforms in the scene.
- For medium-fidelity vessels, let Python publish their pose at a lower rate than the own-ship to reduce load.

Concept to understand

Scaling matters. A simulator that insists on one full hydrodynamic model per vessel may become difficult to extend. In practice, traffic often needs only enough fidelity to affect collision geometry and decision-making.

4. Formalize a simulator core API

Why this matters

The bridge already works, but a platform needs a formal message contract so that adding sensors, controllers, replay, and traffic does not lead to fragile ad hoc JSON packets.

What the API should define

- Time stamps and step counters
- Frame names and coordinate conventions
- Own-ship state
- Actuator commands and actuator states
- Traffic vessel states
- Docking or mission status flags
- Current controller mode such as OPEN, DOCK, DRL_AVOID, or DRL_RECOVER
- Optional sensor data
- Optional debug values such as minimum obstacle distance or predicted path length

Unity Editor steps

- Create a folder Scripts/Networking.
- Create serializable C# data classes for OwnShipStateMessage, TrafficStateMessage, SensorMessage, and ScenarioExportData.
- Replace loosely structured JObject parsing where possible with typed data classes to reduce errors.
- Add a PythonBridgeManager component that owns socket configuration, connection startup, reconnection, and parsing.
- Keep one script for networking and another for applying parsed data to the scene.

Beginner note. A data class is just a C# class whose job is to hold values. It should not contain scene logic. This is one of the simplest ways to keep networking code understandable.

5. Add a simulator manager structure in Unity

Why this matters

As projects grow, logic often becomes scattered across scene objects. A manager structure centralizes orchestration. The manager pattern should not turn into one giant script; use several small managers with clear responsibilities.

Recommended managers and roles

Manager	Responsibility
SimulationManager	Start, pause, reset, fixed-step orchestration, simulation speed factor
ScenarioManager	Load scenario assets, spawn scenario objects, enable or disable route sets and goal zones
TrafficManager	Maintain dynamic traffic vessels and simple vessel AI behaviors
SensorManager	Generate GPS, IMU, AIS, and radar-like outputs and optional debug visualizations
PythonBridgeManager	Socket lifecycle, serialization, receive buffers, message dispatch, reconnect logic
ReplayManager	Load run logs and play them back in Unity
UIManager	Operator UI, mission state, controller mode panel, charts, overlays

Unity Editor steps

- Create one empty GameObject for each manager under _App.
- Attach the corresponding scripts one by one and keep inspector fields explicit.
- For each manager, add serialized references to the objects it owns instead of using FindObjectOfType in many places.
- Start simple: SimulationManager should initially only know whether the simulator is idle, running, paused, or replaying.

6. Separate simulation time from rendering time

Core concept

Rendering and simulation are not the same thing. Rendering is how often Unity draws frames. Simulation is how often the plant, controller, and sensor updates advance time. Good simulators run these on different clocks. This keeps the control loop deterministic and prevents frame-rate changes from changing the physics outcome.

Why your project already points the right way

Your Python side integrates the MMG equations at a fixed time step and sends state messages over TCP. That is already closer to simulator design than to a typical game architecture. The missing step is to make Unity visualize the incoming states cleanly without assuming that every rendered frame equals a new physics step.

Actionable implementation

- Choose one simulation time step first, such as 0.05 seconds to match the existing Python integration.
- Let Python remain the source of truth for the simulation step.
- In Unity, receive new states and store the last and next state so that the vessel can be visually interpolated between them.
- Do not perform control optimization in Update(). Update() is for rendering, input capture, and presentation.
- If you need a Unity-side loop, use FixedUpdate for deterministic manager-side stepping, but still keep Python authoritative if that is your architecture.

Unity Editor and code steps

- Create a script named VisualStateInterpolator.cs and attach it to the own-ship root object or to YachtPoseApplier.
- Store two snapshots: previousReceivedState and latestReceivedState.
- In Update(), compute interpolation alpha from local render time and blend position and yaw between the two states.
- Keep one Inspector toggle to turn interpolation on and off so you can compare raw streamed motion with smoothed motion during debugging.

7. Add sensor realism

Why this matters

Today the controller likely has access to near-perfect world truth. That is fine while validating the core MMG, NMPC, and DRL behavior. A simulator platform should also support perception-limited operation, because real autonomy stacks make decisions from sensor estimates, not from omniscient scene access.

Sensors to add first

Sensor	What it returns	Why it matters
GPS	Position and coarse ground velocity	Baseline navigation and replay alignment
IMU	Heading, yaw rate, acceleration	Stabilization, motion estimation, controller feedback
AIS	Cooperative target positions, headings, speeds	Marine traffic awareness
Radar abstraction	Range, bearing, relative velocity, class or confidence	Obstacle awareness in cluttered marinas

Unity implementation steps

- Create a folder Scripts/Sensors.
- Create one MonoBehaviour per sensor model: GPSSensor.cs, IMUSensor.cs, AISSensor.cs, RadarSensor.cs.
- Attach sensor scripts to child objects under the vessel root, even if the output is abstract.
- Add output rate, noise magnitude, dropout probability, and latency fields in the Inspector.

- Publish all sensor outputs through SensorManager rather than directly from each sensor to Python.

Theory to understand

There is a difference between a physical sensor and a sensor model. In a simulator, the sensor model converts world truth into an imperfect measurement. The measurement may include noise, reduced update rate, missed detections, or delay. These imperfections are often as important to autonomy as the vessel dynamics themselves.

8. Expand the environment model beyond calm water

Why this matters

Low-speed docking becomes significantly harder when current, wind, shallow-water effects, or moving traffic are present. The simulator should allow environmental forcing to be introduced in a controlled way so that robustness and safety margins can be studied systematically.

Recommended environment channels

- Wind force and yaw moment
- Current velocity field or uniform current
- Wave or sea-state preset for visual realism and optional disturbance injection
- Shallow-water correction mode later, once the baseline simulator is stable
- Day, night, and visibility presets for visual and perception studies

Unity Editor steps

- Create ScriptableObject assets for environment presets, such as CalmMorning, HarborWindy, NightCurrent, and TrainingRandomized.
- Add an EnvironmentManager component under _App or make it part of SimulationManager if you want one fewer script at the beginning.
- For visual settings, control Crest parameters, skybox, fog, directional light intensity, and reflection settings from the active preset.
- For physical settings, send wind and current parameters to Python as part of the scenario or run-start message.

Theory to understand

Visual water and physical water are not the same. Crest is excellent for rendering and height queries, but maneuvering truth still belongs on the Python side if you want a clean, controllable scientific model. This distinction helps keep the simulator both beautiful and defensible.

9. Build mission modes instead of one docking task

Why this matters

A simulator platform is most useful when it supports a family of tasks. Your current project already covers approach, avoidance, recover, and docking. These should become formal mission modes so that a controller can be evaluated across a range of objectives.

Core mission modes to add

- Waypoint following
- Open-water transit with traffic
- Approach channel navigation
- Obstacle avoidance and route recovery
- Final berth docking
- Station keeping
- Replay-only analysis mode

Unity implementation steps

- Create a MissionDefinition ScriptableObject with mission type, success conditions, timeout, and optional linked scenario.
- Add mission status fields to UIManager: missionState, elapsedTime, distanceToGoal, alignmentError, collisions, and current controller mode.
- Create visible goal markers and optional gizmos so you can debug mission geometry directly in the Scene view.
- Attach trigger colliders to berth entry zones and keep them on a dedicated layer, for example DockZones.

10. Add evaluation and benchmarking

Why this matters

A simulator becomes a research tool only when results are measured consistently. Screenshots and subjective observations are useful during development, but serious comparison between NMPC, DRL, and hybrid control requires stored metrics.

Metrics to log automatically

- Run identifier, scenario identifier, controller identifier, and random seed
- Success or failure state
- Time to completion
- Number of collisions and minimum obstacle distance
- Final berth position and heading error
- Path length, integrated control effort, and actuator saturation time
- Mode switches between NMPC and DRL
- Environment conditions such as wind and current

Unity and Python implementation steps

- Create a common run folder structure under a Logs directory, with a timestamp and scenario name.
- On the Python side, continue saving time histories of state and control, because that is the authoritative plant log.
- On the Unity side, save a run summary, screenshot thumbnails, and optional replay metadata.
- Add one BenchmarkRunner mode later that iterates over multiple ScenarioDefinition assets automatically.

11. Create a training interface for RL

Why this matters

Training PPO directly inside a rich Unity scene is usually slower and harder to iterate. Your current design—extracting the Unity scene into a Python/Gym environment—is already a practical pattern. The improvement now is to formalize the relationship between the training environment and the simulator.

Recommended structure

- Fast training environment in Python/Gym for high-volume PPO learning
- Validation environment tied to Unity scenario definitions for visual and geometric consistency
- Shared scenario assets or exports so that a training episode can be traced back to a Unity scenario
- Shared metrics so that RL results and simulator results are comparable

Practical steps

- Keep the current scene export pipeline and extend it rather than replacing it.
- Ensure each exported obstacle has an ID, category, pose, and geometric footprint.
- Store docking goals and trigger dimensions in the same export so the RL environment can reconstruct mission geometry precisely.
- Build a validation script that replays a policy result in Unity using the same scenario definition and compares minimum distance, time, and terminal errors.

12. Upgrade Unity into a simulator UI

Why this matters

A platform needs visibility. You should be able to see not only the yacht but also the simulation state, controller state, current mission, traffic contacts, and whether the run is live or a replay. Good UI can save days of debugging.

UI panels to build first

- Simulation panel: Start, Pause, Reset, Run speed
- Mission panel: mission name, state, elapsed time, success criteria
- Vessel panel: x, z, heading, surge speed, sway speed, yaw rate, rudder angle, throttle
- Controller panel: NMPC active, DRL active, supervisor state, mode switches
- Safety panel: minimum obstacle distance, collision count, berth alignment error
- Sensor panel: GPS, AIS targets, radar contact count

Unity Editor implementation steps

- Create a Canvas under UIManager. Use Screen Space - Overlay to start because it is the easiest mode for a first implementation.
- Create panel GameObjects named PanelSimulation, PanelMission, PanelVessel, PanelController, and PanelSafety.
- Add TMP Text elements to each panel. If TextMeshPro is not already in use, import TMP essentials when Unity prompts you.

- Create a simple `UIStatePresenter.cs` script that receives current values from managers and updates the text labels once per frame.
- Keep version 1 of the UI plain and readable. Function beats styling during platform construction.

Beginner note. A prefab is a reusable template for a `GameObject`. A `ScriptableObject` is a Unity asset that stores data. In this simulator, prefabs are best for physical scene elements, and `ScriptableObjects` are best for scenarios, environment presets, and mission definitions.

13. Keep Crest, but use it intentionally

Role of Crest in this simulator

Crest should remain the visual and water-surface component of the simulator. It provides high-quality ocean rendering, height sampling, wakes, and environment appearance. It should not silently replace the Python MMG model as the maneuvering truth unless the simulator is deliberately redesigned around Unity-side hydrodynamics.

Recommended uses

- Ocean rendering and marina water appearance
- Wave-height sampling for buoyancy and visual placement
- Foam, wakes, and propeller wash visuals
- Weather and sea-state presentation through environment presets

Recommended non-uses

- Do not let Crest drive vessel motion instead of Python unless that is an explicit redesign.
- Do not hide dynamic truth in several scripts at once; keep one source of truth for pose.
- Do not mix Unity Rigidbody propulsion with Python-driven MMG pose updates on the same vessel without a very deliberate hybrid design.

Unity Editor steps

- Keep Crest under a dedicated `ThirdParty` or `Water` root in the project hierarchy.
- Place the ocean object and wave controllers under `EffectsRoot` or `EnvironmentRoot`, not under the vessel.
- Expose environment preset parameters such as wind direction, foam intensity, and water appearance, but keep maneuvering disturbances on the Python side unless explicitly coupled.
- If you add visual wakes for traffic vessels, make them optional through a prefab toggle so performance can be scaled.

14. Reorganize folders and modules

Why this matters

Good folders do not make code correct, but bad folders make growth painful. Since the goal is to transform the work into a platform, organizing the project now will save repeated cleanup later.

Suggested Unity structure

Folder	Purpose
--------	---------

Assets/Scenes	Base scenes, test scenes, replay scenes
Assets/Prefabs/Environment	Docks, piers, berths, buoys, landmarks
Assets/Prefabs/Vessels	Own-ship, traffic vessels, sensor-equipped variants
Assets/Prefabs/UI	Panels, labels, widgets
Assets/Scripts/Core	Managers and orchestration
Assets/Scripts/Networking	TCP bridge, message classes, serializers
Assets/Scripts/Sensors	GPS, IMU, AIS, radar abstractions
Assets/Scripts/Scenario	Scenario loaders, exporters, route definitions
Assets/ScriptableObjects	Scenarios, missions, environment presets, controller presets
Assets/ThirdParty	Crest and external tools

Suggested Python structure

- `sim_core` for top-level simulator orchestration
- `dynamics` for MMG and actuator models
- `controllers` for NMPC, PPO wrapper, PID, and supervisor
- `missions` for mission logic and success conditions
- `scenarios` for exported marina and route data
- `sensors` for noisy measurement models
- `logging` for run summaries and time histories
- `evaluation` for benchmark scripts and report generation
- `gym_envs` for training environments

15. Define the practical development roadmap

Roadmap philosophy

The best roadmap for your situation is incremental. Because this is your first serious Unity project, the safest approach is to stabilize architecture before adding high visual complexity or advanced sensor simulation. Each phase should end with something runnable and testable.

Phase-by-phase roadmap

Phase	Name	Main objective	Concrete deliverables
0	Stabilize the baseline	Preserve the current project and make it safe to extend	Backup project, version control, cleaned scene roots, verified Python bridge, verified coordinate mapping
1	Platform cleanup	Introduce managers and folder structure	_App managers, WorldRoot hierarchy, typed networking classes, basic SimulationManager

2	Scenario system	Move from one scene to reusable scenarios	ScenarioDefinition assets or JSON, prefab-based obstacles, SceneManager load/reset flow
3	Multi-vessel and missions	Support traffic and reusable tasks	TrafficManager, vessel tiers, mission assets, berth triggers, route markers
4	Sensors and UI	Make autonomy less omniscient and more observable	SensorManager, GPS/IMU/AIS/radar abstractions, operator UI panels, debug overlays
5	Evaluation and replay	Turn runs into experiments	Run logs, replay mode, benchmark summaries, controller comparison flow
6	Polish and expansion	Improve realism and usability	Environment presets, Crest wake polish, batching tools, domain-randomization hooks

Detailed implementation plan by phase

Phase 0 — Stabilize the baseline

- Duplicate the current Unity project folder and keep one untouched archival copy.
- Create a Git repository if one does not already exist. Commit the current working state before restructuring anything.
- Confirm that the current Python-to-Unity link still moves the yacht correctly and that the coordinate mapping is documented in one place.
- Open the main scene and reorganize the Hierarchy without changing behavior: _App, WorldRoot, MarinaRoot, TrafficRootStatic, TrafficRootDynamic, and UIRoot.

Phase 1 — Platform cleanup

- Create the manager GameObjects and attach empty scripts first.
- Replace direct scene searches with Inspector references where practical.
- Move networking classes into Scripts/Networking and convert ad hoc parsing to typed message models.
- Add a simple simulation state enum: Idle, Running, Paused, Replay.

Phase 2 — Scenario system

- Create prefabs for all repeated environment objects.
- Implement ScenarioDefinition and load one scenario asset successfully.
- Add reset logic that despawns dynamic traffic and respawns it cleanly.
- Preserve the JSON export flow so the same scenario can be reconstructed in Python.

Phase 3 — Multi-vessel and missions

- Add at least one dynamic traffic vessel on a waypoint route.
- Create mission definitions for ApproachOnly, AvoidAndRecover, and Dock.
- Display mission state in the UI and log mission result at the end of every run.

Phase 4 — Sensors and UI

- Create the four core sensor scripts and expose their parameters in the Inspector.
- Add a SensorManager that gathers outputs and provides them as one bundle.
- Build a simple operator HUD using Canvas and TextMeshPro.

Phase 5 — Evaluation and replay

- Save one log file per run with state, control, and events.
- Create a ReplayManager that can read a saved run and drive the Unity view without a live Python connection.
- Implement a benchmark scene or menu that compares NMPC and Hybrid over several scenarios.

Phase 6 — Polish and expansion

- Add environment presets for calm, windy, and night scenarios.
- Improve water visuals and wake effects, but keep them optional for performance.
- Add domain-randomization hooks for future RL training work.
- Only after the platform is stable, consider moving to a higher-detail scene or adding pipeline experiments in a separate branch.

Part II — Unity Editor implementation steps

This part translates the architecture into concrete steps inside the Unity 6000.3.9f1 editor. The goal is not to write every line of code immediately, but to help you know exactly what to click, what to create, and in what order to build the platform.

A. Create a safe working branch of the project

- Close Unity and duplicate the full project folder in the file explorer. Name the copy something clear, such as YachtSimulator_v2.
- Open the copied project in Unity Hub.
- Let Unity upgrade project metadata if needed, but do not change render pipeline or package choices during this first restructuring step.
- Open the main marina scene and save it as a new scene name, such as Marina_Simulator_Base.

B. Build the root hierarchy in the Scene view

- In the Hierarchy panel, right-click and choose Create Empty. Name the object _App. Press the gear icon in the Inspector and choose Reset.
- Create another empty object named WorldRoot.
- Under _App, create child empty objects named SimulationManager, ScenarioManager, TrafficManager, SensorManager, PythonBridgeManager, ReplayManager, and UIManager.
- Under WorldRoot, create child empty objects named EnvironmentRoot, MarinaRoot, TrafficRootStatic, TrafficRootDynamic, SensorVisualsRoot, and EffectsRoot.

- Drag existing docks, piers, obstacles, and landmark objects into these folders so the scene becomes readable immediately.

C. Turn repeated objects into prefabs

Prefabs are reusable templates. In practice, this means you build one buoy once and then reuse it many times while keeping a consistent script setup and collider configuration.

- Create project folders Prefabs/Environment, Prefabs/Vessels, and Prefabs/UI if they do not already exist.
- Select one dock section in the Hierarchy and drag it into Prefabs/Environment. Unity creates a blue prefab asset.
- Repeat this for a pier, a berth marker, a buoy, and a static obstacle vessel.
- When you need a new copy in the scene, drag the prefab from the Project window into the Hierarchy rather than duplicating an arbitrary scene object.

D. Create ScriptableObjects for scenario and mission data

ScriptableObjects are Unity assets that store data without needing to be attached to scene objects. They are ideal for scenarios, mission settings, and environment presets.

- Create a folder Assets/ScriptableObjects.
- Add subfolders named Scenarios, Missions, and EnvironmentPresets.
- Create C# scripts ScenarioDefinition.cs, MissionDefinition.cs, and EnvironmentPreset.cs.
- Use CreateAssetMenu attributes in each script so Unity shows a Create menu for the asset type.
- After the scripts compile, right-click in each ScriptableObjects folder and create the corresponding assets.
- Fill in scenario data in the Inspector: spawn pose, goal pose, active traffic, environment preset, and controller preset.

E. Create the manager scripts

- Create a folder Scripts/Core.
- Create empty scripts named SimulationManager.cs, ScenarioManager.cs, TrafficManager.cs, SensorManager.cs, PythonBridgeManager.cs, ReplayManager.cs, and UIManager.cs.
- Attach each script to the GameObject with the same name under _App.
- In each manager, start with only serialized references and clear TODO comments. Avoid writing large amounts of logic until the object structure is stable.
- Use [Header] attributes in scripts so the Inspector stays organized and beginner-friendly.

F. Clean up the Python bridge

The project already has PythonTCPReceiver, MMGStateReceiver, UnityMainThreadDispatcher, and YachtPoseApplier patterns. The platform version should keep the same logic but reorganize it into clearer responsibilities.

- Move networking scripts into Scripts/Networking.
- Create one bridge owner, ideally PythonBridgeManager, that handles connect, disconnect, and receive loop setup.
- Keep one dispatcher for thread-safe execution on Unity's main thread.

- Let vessel-specific scripts only apply pose, actuator state, or visual effects. They should not own socket management.
- Document the coordinate mapping in one script comment block and one project note so the sign convention never becomes ambiguous.

G. Prepare the vessel prefab structure

A clean vessel hierarchy makes later work much easier. A practical pattern is one root pivot for pose, one visual mesh child, and several child transforms for actuators and sensors.

- Create a root object named OwnShipRoot.
- Under it, create ShipPivot, VisualMesh, Sensors, Actuators, and Debug sub-objects.
- Place the yacht mesh under VisualMesh.
- Place rudder pivot and propeller pivots under Actuators.
- Place future GPS, IMU, AIS, and radar objects under Sensors.
- Attach YachtPoseApplier or an equivalent pose script to the root or ShipPivot only.

H. Build the first scenario loader

- In ScenarioManager, add a serialized field for the active ScenarioDefinition asset.
- Add references for parent transforms such as TrafficRootStatic and TrafficRootDynamic.
- Write a LoadScenario() method that clears existing spawned traffic, instantiates the required prefabs, places the own-ship at the scenario start pose, and places the goal marker at the goal pose.
- Add a ResetScenario() method that simply calls LoadScenario() again.
- Expose buttons later through the UI to invoke LoadScenario and ResetScenario.

I. Add route markers for moving traffic

- Create an empty object called Route_01.
- Under it, create child empties named WP_00, WP_01, WP_02, and so on.
- Place them along the path you want a traffic vessel to follow.
- Create a simple RouteFollower script for a background or medium-fidelity vessel that moves toward the current waypoint and then advances to the next.
- Save this route definition in a route asset or keep it under a dedicated scene root until you formalize route data.

J. Build the first UI

- Select UIManager and create a Canvas as its child.
- Create a Panel object anchored in a corner of the screen.
- Add TextMeshPro text elements for mode, speed, heading, rudder, throttle, and minimum obstacle distance.
- Write a UIStatePresenter script that reads values from the managers and refreshes the text every frame.
- Do not wait for a perfect visual design. A simple readable debug UI is enough at first.

K. Add a replay path early

- Create a folder for logs outside or inside the project, depending on your workflow.
- Create a ReplayManager that can read one saved file and push state frames to the same pose application path used by the live bridge.

- Add a bool in SimulationManager to indicate whether the simulator is in Live or Replay mode.
- This reuse of the same pose-application path is important because it keeps replay visually trustworthy.

Part III — Final checklists and recommended next actions

A. First 2-week implementation checklist

- Create a safe project copy and commit the baseline to version control.
- Rebuild the hierarchy into _App and WorldRoot roots.
- Create manager GameObjects and empty scripts.
- Move networking scripts into a dedicated folder and define a bridge owner.
- Convert at least three repeated environment objects into prefabs.
- Create one ScenarioDefinition asset and load it through ScenarioManager.
- Add one simple operator UI panel with live vessel state.
- Verify that the own-ship still moves correctly from Python after the cleanup.

B. First 6-week milestone

- At least one reusable scenario asset is working.
- At least one dynamic traffic vessel follows a route.
- Mission states are visible in the UI.
- Logs are saved for each run.
- Replay mode can play a saved run in Unity without live Python streaming.
- NMPC and Hybrid can be run on the same scenario for comparison.

C. Common beginner mistakes to avoid

- Do not attach random logic to many scene objects just because it works once. Decide which manager owns the feature.
- Do not mix multiple sources of truth for vessel motion. Choose Python or Unity physics, not both at once for the same vessel.
- Do not hard-code scenario geometry inside controller scripts.
- Do not let coordinate conversion appear in several scripts with different signs.
- Do not delay logging and replay. They become far harder to add after the project grows.

D. Recommended sequence of future expansion

- Stabilize architecture first.
- Then add scenario loading and reset.
- Then add multi-vessel support and mission definitions.
- Then add sensors and UI.
- Then add replay and benchmarking.
- Only after the simulator platform is stable, increase visual scope or environment complexity.

Bottom line. Your current project already contains the hard scientific core of a simulator platform: MMG dynamics, optimal control, learning-based avoidance, a supervisor, scenario extraction, and a Unity co-simulation viewer. The work ahead is mainly architecture, tooling, and disciplined scene organization. If you build those layers carefully, the project can grow from a defended thesis into a reusable Unity-based

marine autonomy simulator platform.

Project-aligned reference notes

This document is aligned with the uploaded thesis and presentation, which describe the Unity–Python co-simulation architecture, the layered static and dynamic obstacle setup, the scenario export pipeline, and the MMG–NMPC–PPO hybrid architecture used in the project. Those materials also make clear that Unity was intentionally used as a rendering and scenario engine while vessel dynamics remained in Python, which is the central design assumption preserved in this guide.

Suggested companion artifacts to create next

- A Unity scene hierarchy diagram
- A Python–Unity message schema specification
- A ScenarioDefinition and MissionDefinition asset template
- A SimulationManager and ScenarioManager UML or class map
- A benchmark protocol comparing NMPC, DRL, and Hybrid across fixed scenarios

End of guide